SPARKLETEAM: Towards Recursive Self Home Improvement

(and the Superstitions It Learned on the Way)

Dave Rauchwerk
Independent Researcher, @_elevenarms

Claude
Anthropic*

Abstract

How best to load a dishwasher is a combinatorial, physical problem with an unusually honest reward: a simulated wash decides which items come out clean [8]. We use it as a testbed for SPARKLETEAM, a self-improving multi-agent system built only from open, inexpensive models. A deliberately weak loader (Nemotron-3.5-Super-VL, thinking off) is surrounded by a planner, a vision critic, a coach and a librarian, all open Nemotron models on Crusoe serverless inference. Between generations the team rewrites a typed genome of its own prompts, playbook, tool policy and structure, and a deterministic gatekeeper keeps an edit only if it beats the incumbent on paired, fresh tasks. The system works end to end, and quantifies a known physical effect: a cup stood mouth up keeps half its grime. A first held-out exam showed $+17.8$ pp; a 12-drill replication showed -0.5 pp (CI -13 to $+11$). We trace the gap to learned superstitions: the team accepted one rule that is real physics and one that is noise, and the loader's sampling noise made them indistinguishable. We give design rules to fix this.

1 Introduction

Recursive self-improvement is usually discussed for systems that rewrite their own weights. A humbler, testable version keeps the model fixed and lets the system rewrite the *scaffold* around it: prompts, memory, tools and the division of labour [20, 66, 68]. Such loops are hard to trust: the reward is often an LLM judge, which can be flattered [71], a loop optimising a proxy finds its seams [27, 33, 52], and gains are usually measured where they were selected.

We take the problem of loading a dishwasher, using DishwasherBench, Composo's open (MIT) benchmark [8]. It models one real machine (Bosch SMV4HAX40G): items are placed with deterministic rigid-body physics (Jolt) through 13 MCP tools, then a GPU wash simulation (WebGPU via Dawn) of the Eco 50 programme with one detergent capsule, calibrated on washes of the real machine (parameter set `anchored-v8.owner-capsule`), decides which items are clean against a threshold derived from the IEC 60436 scale [6, 12, 21]. An item scores its points only if it comes out clean. The reward is *objective* (points over points available), *deterministic* (a load always settles and washes the same way, and every episode is verifiable by replay) and *unfakeable*: a bowl mouth-up under a dead spot of the spray arm stays dirty however eloquently the agent describes it. It is also hard: not everything fits, so the player chooses what to load as well as how; jets reach only part of the load [43, 44]; and choosing a subset under capacity is a knapsack over strongly NP-hard 3D packing [24, 34]. Household benchmarks score symbolic goals [45, 49, 51, 55], and prior dishwasher-loading work scores grasp success or agreement with a human preference [22, 23, 30, 56]; to our knowledge none scores whether the items come out clean, the gap DishwasherBench fills.

The benchmark, simulator, calibration, levels and harness are Composo's work [8]. Ours is SPARKLETEAM, the self-improving team, with its memory, Gatekeeper, exam protocol and visualiser (Fig. 1), plus cosmetic water, watch and turntable changes to the benchmark's web client that touch neither simulation nor scoring. We asked whether cheap open models can lift a weak loader by rewriting the system around it, keeping only changes that measurably help. The honest answer is: not yet, and we can show why. Contributions:

- SPARKLETEAM, a team whose rewritable state is a typed, versioned *genome*, edited only through six typed operations, with the judging roles (memory accounting, acceptance) in deterministic code: the team can change how it plays, never how it is graded.
- A paired acceptance rule, a three-layer memory with shrunken Bayesian confidence, and a held-out exam with a replication protocol (12 drills $\times$ 2 repeats).
- A diagnosis of *learned superstitions*, with a noise analysis, and design rules for self-improving loops on noisy weak models: more pairs and repeats, a statistical acceptance test instead of a fixed margin, and physics falsification of each learned rule before it is kept.

2 Related Work

Scaffold self-improvement. STOP recursively improves the improver around a fixed LM [66]; ADAS's meta-agent programs new agents [20]; the Darwin Gödel Machine keeps self-edits that validate, in an open-ended archive [68]; Gödel Agent and SICA rewrite their own logic or code [47, 63]. AFlow searches workflows with MCTS and lets cheap models match expensive ones [67]; GPTSwarm optimises prompts and agent connectivity [72]; FunSearch and AlphaEvolve gate LLM proposals with automatic evaluators [36, 48]. Our Coach is a typed, constrained meta-agent in this line.

Prompt and program optimisation learns from scored histories, LLM mutation, compiled programs and textual feedback [7, 13, 16, 25, 40, 62, 64]; GEPA's reflection on traces is closest to our Critic→Coach loop [1].

Memory and multi-agent systems. Agents keep reflections, cross-task insights, skills and workflows [32, 41, 42, 50, 59, 60, 69]; unlike self-refinement [31], our lessons are counted against physics. Role-based teams [19, 28, 46, 61] and debate [10] follow fixed processes; ours edits its own structure. Voting and verified repeated sampling trade compute for parameters [4, 29, 53]; our verifier is too slow to call per action.

Search over loads. Loading is constrained packing with a physical, black-box objective [58, 70]; genetic algorithms [15, 18], CMA-ES over poses [17], quality-diversity [35], Bayesian optimisation [54] and MCTS over placement sequences [5, 26] are natural per-instance optimisers. We do not search loads directly: every evaluation costs a wash, and the product is *one load*, not a better loader, so nothing transfers to a new set of items.

3 Method

The team (Table 1, Fig. 2). In SPARKLETEAM the Loader plays through the standard DishwasherBench harness tools [8], with renders as at most one image per request (the newest). Optionally a Planner turns the level card and a memory brief into a per-item plan (rack, stance, what to leave out). After each practice episode the Critic gets the item-by-item outcome (rack, stance, clean, residue, retained water), plan, playbook and two renders of the final load, and writes up to five

*Built at the Crusoe $\times$ Cerebral Valley Recursive Agents Hackathon, 17 October 2026. Code: `https://github.com/elevenarms/RSHI`.

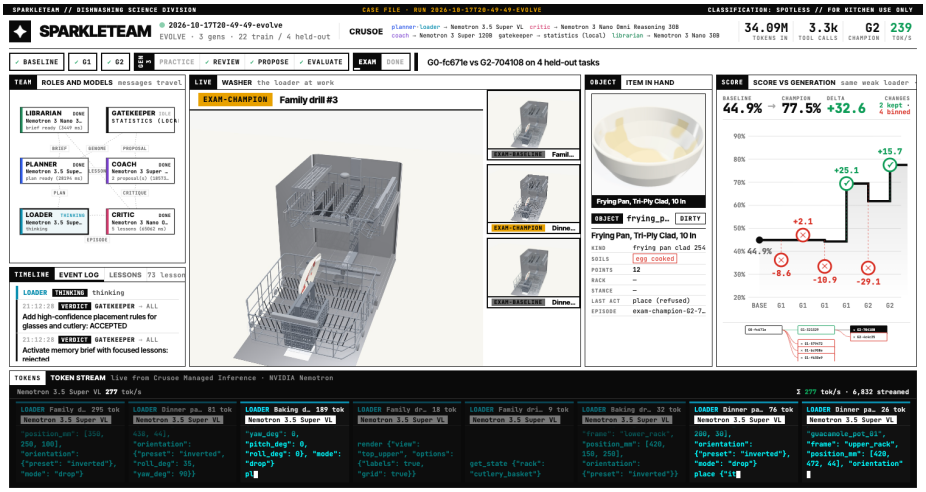


Figure 1: The Studio, the live visualizer of SPARKLETEAM, during the exam of the final run: TEAM (the roles as a graph, each with its Nemotron model), TIMELINE (events and Gatekeeper verdicts), LIVE (the loader at work in the DishwasherBench simulation), OBJECT (the item in hand), SCORE (score vs. generation with accepted and rejected proposals and the genome lineage; its percentages are *validation*-drill means, not exam scores) and TOKENS (8 loader lanes streaming live from Crusoe). Screenshot from a replay of the run's event log; because the game servers had shut down, the LIVE washer view and the object turntable show representative stills rather than that moment's frames (the turntable shows a bowl while the inspector lists a frying pan).

Role	Model (open, on Crusoe)	Job
Loader	Nemotron-3.5-Super-VL (no thinking)	plays, 13 tools
Planner	Nemotron-3.5-Super-VL (low effort)	per-item plan
Critic	Nano-Omni-Reasoning-30B (thinking)	post-mortem
Coach	Nemotron-3-Super-120B-A12B	genome edits
Librarian	Nemotron-3-Nano-30B-A3B + code	memory, brief
Gatekeeper	deterministic statistics	accept/reject

Table 1: Roles and models [37–39].

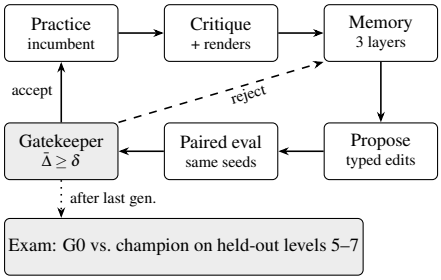


Figure 2: One generation of the loop. Rejections (title, hypothesis, per-task deltas) are logged and shown to the Coach.

tagged lessons with a polarity. The Coach proposes edits; the Librarian condenses memory into a brief.

Genome and typed edits. A genome is an immutable record of everything the team may rewrite: three prompts (Planner, loader preamble, Critic); a playbook of at most 25 tagged rules with evidence links to lessons; team switches (`usePlanner`, `useMemoryBrief`, `preflightCheck`, which refuses `start_cycle` before a read `check_load`, and the brief's lesson count); and loader settings (reasoning effort, images, denied tools, turn limit). The Coach can change it only by `add_rule`, `edit_rule`, `remove_rule`, `set_prompt`, `set_team` and `set_loader`. A proposal carries a title and a falsifiable hypothesis, changes one coherent thing, and is shown past rejections; ids hash content and parent, so lineage is diffable. G0 is the bare loader: no planner, brief or preflight, an empty playbook.

Three-layer memory. The *episodic* layer keeps every training episode item by item; the *semantic* layer keeps lessons (from the Critic, and observed from placement statistics), deduplicated on merge, with support s and contradiction c counts; the *procedural* layer keeps every genome and decision. Confidence is a Beta(1,1) posterior mean shrunk towards ignorance while evidence is thin:

$$p = \frac{s+1}{n+2}, \quad \kappa = \frac{1}{2} + \left(p - \frac{1}{2}\right)\frac{n}{n+K}, \quad n = s+c,\ K = 3, \tag{1}$$

so one supporting episode gives 0.54, five of five 0.72, twenty of twenty 0.90. A level's brief lists smoothed clean rates $(k+1)/(m+2)$ of its kinds per rack and stance, then lessons with $\kappa \geq 0.5$ ranked by $\kappa \cdot \text{relevance} \cdot \log(1+n)$. Exam episodes are stored apart and never read back.

Gatekeeper. Candidate C and incumbent I play the same fresh validation drills with the same seeds. Let P be the drills both played validly and x_t the share on drill t; then

$$\bar{\Delta} = \frac{1}{|P|}\sum_{t \in P}\left(x_t^C - x_t^I\right), \qquad \text{accept} \iff \bar{\Delta} \geq \delta, \tag{2}$$

and the decision is void if $|P| < 3$. Agent failures (turn or token limit, nothing loaded) score what they scored; *invalid* episodes (provider or harness errors) are replayed on the same task and seed up to twice, and drills still unpaired are dropped for both genomes.[1] Pairing matters because episodes vary enormously (G0 scored 88% on one Tutorial drill and 0% on a Weeknight drill); with deterministic physics it cancels drill difficulty. The accepted candidate with the largest $\bar{\Delta}$ becomes the incumbent; its validation episodes are the next practice set.

Exam: what "held-out" means. Training and validation drills come from DishwasherBench course levels 1–4 (tutorial, lunch for four, weeknight dinner, breakfast), the exam from levels 5–7 (mixed family, baking day, dinner party) [8]. These are new task families with kinds and soils training never showed (baby bottle, spouted beakers, lunch tubs, baking tray, tin, platter; burnt sugar, overnight soils). Nothing is learned from the exam.

4 Experimental Setup

All models run on Crusoe Managed (serverless) Inference via an OpenAI-compatible API, with a client-side rate limiter and byte-stable prompts for prefix caching (83–90% of input tokens hit the cache in smoke runs). **Choosing a weak loader.** A saturated loader leaves the Gatekeeper nothing to measure. On one 12-item drill, DeepSeek-V4.1-Flash and Qwen3.8-27B scored 100%, gpt-oss-120b 49%, Nano-30B with thinking 41% at ∼6× the wall time (without, it looped);

[1] In a pilot, HTTP 429s were first scored as agent failures and a generation was voided.

G0 vs. champion G2-704108	4-drill exam	12-drill replication
Paired drills (valid pairs)	4	12×2 repeats (24)
Mean share, G0 $\rightarrow$ champion	46.0% $\rightarrow$ 63.7%	47.0% $\rightarrow$ 46.6%
Δ (pp)	$+17.8$	-0.5
95% CI (bootstrap)	n/a	$[-13, +11]$
Wins / losses / ties	3 / 1 / 0	12 / 10 / 2
Sign test p	0.31 (one-sided)	0.83 (Wilcoxon 0.76)

Table 2: Held-out results, same genomes, fast tier. The first exam's gain did not replicate.

G	Proposal (Coach's title, abridged)	Inc.	Cand.	W/L	
1	Enable Planner	.449	.363	1/2	rej.
1	Memory Brief, expanded lessons	.449	.471	1/2	rej.*
2	Enable preflight check	.443	.334	1/2	rej.
2	Avoid poor-performing bowl placement	.443	.695	3/0	**acc.**
3	Memory brief, focused lessons	.618	.327	1/2	rej.
3	Rules for glasses & cutlery	.618	.775	2/1	**acc.**

Table 3: Decisions: validation share of incumbent and candidate on 3 paired drills (fresh seeds per generation). *+0.129, accepted, under the corrected rule (fn. 2).

Nemotron-3.5-Super-VL without thinking scored 21% (all it loaded clean) and is our loader. A Lightning pilot was throttled by its shared endpoint to $\sim$5 calls/min.

Protocol. A drill is 12 items of a DishwasherBench level drawn by seed, cutlery in proportion. The evolution run (`2026-10-17T20-49-49-evolve`) has 3 generations: 4 baseline practice drills, 2 proposals and 3 paired fresh validation drills per generation, $\delta = 0.03$, a 40-turn cap and 8 episodes in flight. G0 and the champion then sat a 4-drill held-out exam and, separately, a *replication*: 12 held-out drills from levels 5–7, each played twice by both genomes. Washes use the fast tier (4 mm blocks rather than 2 mm), whose verdicts agree with the official tier's on 91% of the standard load and 97% of pieces on the course levels [8]. G0 scored 39% in practice, loading few items but nearly all of them clean: it fails at commitment, not physics.

5 Results

Evolution. The system ran end to end: the Gatekeeper accepted 2 of 6 candidates (Table 3), both memory-derived playbook rules, and rejected every structural edit (planner, memory brief, preflight).[2] The champion differs from G0 by three rules: R1.1 "Do not place `cereal_bowl_vitrelle_159` on upper_rack/mouth_down_tilted_45" (+25.1 pp in validation), and R2.1 "Glasses and tumblers: upper_rack, mouth_down" with R2.2 "Cutlery: cutlery_basket, upright" (+15.7 pp).

Exam and replication. On the first 4-drill exam the champion scored 63.7% against 46.0% (+17.8 pp, 3 of 4 wins). It did not replicate: on 12 held-out drills played twice (24 valid pairs) the champion scored 46.6% against 47.0%, $\Delta = -0.5$ pp, 95% CI $[-13, +11]$, sign test $p = 0.83$ (Table 2). In an ablation no single rule had a significant effect ($p \geq 0.39$); the best variant was the champion *without* R1.1 (+8 pp, n.s.).

Finding 1 (quantified, not discovered). A cup stood mouth up keeps half its grime. In 108 deterministic fast-tier washes of every hollow kind in the course levels, each item alone (73 classifiable by resting stance), drinkware and ramekins were always clean mouth down and never mouth up (Table 4). Bowls mouth up still washed clean (8/8) but kept 1.17 L of water each (CI 0.66–1.78); one mixing bowl held 2.9 L and starved the pump. Over 15 hollow items seen in both stances, mouth down adds 60 pp to P(clean) (CI 33–87; odds ratio $\approx$130 with a 0.5 correction); no tilt from 0 to 45° beats another. An official-tier spot check is starker: a mug and a tumbler mouth up kept 90% and 89% of their soil. The effect is long known (phenomenon P01 [8]); the number is new, for one machine and programme, the fast tier, items alone and small n per kind.

Drinkware, ramekins (alone)	P(clean)	Soil left	Water
Mouth down (0–45°)	100% (26/26)	0.000	0 ml
On its side	25% (2/8)	0.038	1 ml
Mouth up	0% (0/11)	0.488 [.28,.71]	224 ml [140,302]

Table 4: Finding 1: hollow items washed alone, by resting stance (95% CIs).

Learned superstitions. We report, with appropriate gravity, that our team became superstitious. R2.1 is real physics: it is Finding 1. R1.1 is not. It rested on one dirty bowl in two loads, and the controlled sweep washed every mouth-down bowl clean, 45° included. The memory had never seen a mouth-up hollow item, so it learned "mouth down" from successes alone; and its largest real losses were never targeted: bowls left out (21 of 21 times, though every bowl kind washes clean mouth down) and pots, which mouth up kept 95–100% of their soil and 2–7 L of water. The Gatekeeper could not tell the true rule from the superstition, because the loader's sampling noise swamps the margin: the same genome on the same drill differed by 0.20 share on average between repeats (up to 0.80; one drill scored 0.000 and 0.803). With a per-drill SD near 0.3, three or four paired drills cannot resolve a 3 pp margin, and even 24 pairs resolve only about $\pm$12 pp.

Cost. 22.7 min of evolution plus about 7 min of exam; the loader used 38.24M input tokens (83.7% cached). Cost: about $0.12 for the Critic, Coach and Librarian at listed prices; Super-VL's price is not listed (at Super-120B's rates the loader would cost about $7.7).

6 Discussion and Limitations

The architecture held: physics scored, code decided, models proposed [2, 71], and full logs let us find out *why* a gain vanished. What failed is the statistics of selection [3, 11, 14]: a fixed margin on three drills accepted one true rule and one superstition with equal confidence. Three design rules follow for self-improving loops on noisy weak models. (i) Many more paired drills, with repeats per drill. (ii) A statistical acceptance test (a sequential test [57] or a confidence bound [9]) instead of a fixed margin. (iii) *Falsify the lesson, not just score the genome*: test each learned rule with cheap controlled washes before it enters the playbook, which a deterministic simulator makes possible. *Limitations*: one machine, programme, detergent and the fast tier; one evolution run; 24 pairs bound the effect only to $\pm$12 pp.

7 Conclusion

A team of open models can rewrite its own system and keep only what measurably helps; ours learned one true rule and one superstition and could not tell them apart. *We propose* (not done) the next recursion: a scaffold can only steer a noisy policy, so distil the team's best trajectories, logged token by token, into the weak loader by LoRA SFT on Crusoe Serverless Fine-Tuning, which supports Nemotron-3.5-Lightning;[3] a Self-Serve Deployment would also escape the throttling. A seed set (7 episodes, 40 examples, $\approx$0.53M tokens) is already exported from the logs; it is far too small, so the natural next loop is expert iteration, STaR-style [65]: run many episodes, keep the best, fine-tune, repeat. The aim is lower variance between repeats, so the paired test gains power: the scaffold discovers, the Gatekeeper verifies, fine-tuning consolidates. Recursive self-(home-)improvement needs a scientist on the team.

Acknowledgements. We thank Composo for releasing DishwasherBench (MIT), Crusoe for inference, NVIDIA for Nemotron.

[2] In generation 1, G0's Weeknight attempt that blocked the door (0 of 87) was wrongly replayed as invalid. Under the rule as since fixed, the Memory Brief would have been accepted at +0.129; the champion's lineage is unaffected.

[3] `https://docs.crusoecloud.com/serverless-fine-tuning/available-models`

References

[1] L. A. Agrawal, S. Tan, D. Soylu, et al. GEPA: Reflective prompt evolution can outperform reinforcement learning, 2025. arXiv preprint arXiv:2507.19457.

[2] D. Amodei, C. Olah, J. Steinhardt, et al. Concrete problems in AI safety, 2016. arXiv preprint arXiv:1606.06565.

[3] Y. Benjamini and Y. Hochberg. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B (Methodological)*, 57 (1):289–300, 1995.

[4] B. Brown, J. Juravsky, R. Ehrlich, et al. Large language monkeys: Scaling inference compute with repeated sampling, 2024. arXiv preprint arXiv:2407.21787.

[5] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, and S. Colton. A survey of Monte Carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, 2012.

[6] CENELEC. EN 60436:2020 electric dishwashers for household use – methods for measuring the performance (incorporating IEC 60436:2015+a1:2020). European standard; successor of EN 50242, 2020.

[7] C.-A. Cheng, A. Nie, and A. Swaminathan. Trace is the next AutoDiff: Generative optimization with rich feedback, execution traces, and LLMs, 2024. arXiv preprint arXiv:2406.16218.

[8] Composo. DishwasherBench: A dishwasher-loading game on a physical simulation of one real machine. Software, version 0.1.0. `https://github.com/composo-ai/DishwasherBench`, Oct. 2026. Released 2026-10-12. MIT License. `https://composo.ai`.

[9] R. Dror, G. Baumer, S. Shlomov, and R. Reichart. The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1383–1392, 2018.

[10] Y. Du, S. Li, A. Torralba, et al. Improving factuality and reasoning in language models through multiagent debate, 2023. arXiv preprint arXiv:2305.14325.

[11] C. Dwork, V. Feldman, M. Hardt, T. Pitassi, O. Reingold, and A. Roth. The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349(6248):636–638, 2015.

[12] B. Eilts, A. Klingshirn, and D. Boursillon. Dishwasher round robin test 2022/2023: Final report. Technical report, University of Applied Sciences Albstadt-Sigmaringen, Sept. 2023. Sponsored by APPLiA and CENELEC.

[13] C. Fernando, D. Banarse, H. Michalewski, et al. Promptbreeder: Self-referential self-improvement via prompt evolution, 2023. arXiv preprint arXiv:2309.16797.

[14] A. Gelman and E. Loken. The garden of forking paths: Why multiple comparisons can be a problem, even when there is no "fishing expedition" or "p-hacking" and the research hypothesis was posited ahead of time. Working paper, Department of Statistics, Columbia University, 2013.

[15] D. E. Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, Reading, MA, 1989. ISBN 0201157675.

[16] Q. Guo, R. Wang, J. Guo, et al. EvoPrompt: Connecting LLMs with evolutionary algorithms yields powerful prompt optimizers, 2023. arXiv preprint arXiv:2309.08532.

[17] N. Hansen and A. Ostermeier. Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9(2):159–195, 2001.

[18] J. H. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, Ann Arbor, MI, 1975. ISBN 0472084607.

[19] S. Hong, M. Zhuge, J. Chen, et al. MetaGPT: Meta programming for a multi-agent collaborative framework, 2023. arXiv preprint arXiv:2308.00352.

[20] S. Hu, C. Lu, and J. Clune. Automated design of agentic systems, 2024. arXiv preprint arXiv:2408.08435.

[21] International Electrotechnical Commission. IEC 60436:2025 electric dishwashers for household use – methods for measuring the performance. IEC standard; corrected version 2026-04, 2025.

[22] V. Jain, Y. Lin, E. Undersander, Y. Bisk, and A. Rai. Transformers are adaptable task planners. In *Proceedings of the 6th Conference on Robot Learning (CoRL)*, volume 205 of *Proceedings of Machine Learning Research*, pages 1011–1037, 2023.

[23] I. Kapelyukh and E. Johns. My house, my rules: Learning tidying preferences with graph neural networks. In *Proceedings of the 5th Conference on Robot Learning (CoRL)*, volume 164 of *Proceedings of Machine Learning Research*, 2022.

[24] H. Kellerer, U. Pferschy, and D. Pisinger. *Knapsack Problems*. Springer, Berlin, Heidelberg, 2004.

[25] O. Khattab, A. Singhvi, P. Maheshwari, et al. DSPy: Compiling declarative language model calls into self-improving pipelines, 2023. arXiv preprint arXiv:2310.03714.

[26] L. Kocsis and C. Szepesvári. Bandit based Monte-Carlo planning. In *Machine Learning: ECML 2006*, volume 4212 of *Lecture Notes in Computer Science*, pages 282–293. Springer, 2006.

[27] V. Krakovna, J. Uesato, V. Mikulik, et al. Specification gaming: the flip side of AI ingenuity. DeepMind blog, 2020.

[28] G. Li, H. A. A. K. Hammoud, H. Itani, et al. CAMEL: Communicative agents for "mind" exploration of large language model society, 2023. arXiv preprint arXiv:2303.17760.

[29] J. Li, Q. Zhang, Y. Yu, et al. More agents is all you need, 2024. arXiv preprint arXiv:2402.05120.

[30] Y. Lin, A. S. Wang, E. Undersander, and A. Rai. Efficient and interpretable robot manipulation with graph neural networks, 2021. arXiv preprint.

[31] A. Madaan, N. Tandon, P. Gupta, et al. Self-refine: Iterative refinement with self-feedback, 2023. arXiv preprint arXiv:2303.17651.

[32] B. P. Majumder, B. Dalvi Mishra, P. Jansen, et al. CLIN: A continually learning language agent for rapid task adaptation and generalization, 2023. arXiv preprint arXiv:2310.10134.

[33] D. Manheim and S. Garrabrant. Categorizing variants of Goodhart's law, 2018. arXiv preprint arXiv:1803.04585.

[34] S. Martello, D. Pisinger, and D. Vigo. The three-dimensional bin packing problem. *Operations Research*, 48(2):256–267, 2000.

[35] J.-B. Mouret and J. Clune. Illuminating search spaces by mapping elites, 2015.

[36] A. Novikov, N. Vũ, M. Eisenberger, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery, 2025. arXiv preprint arXiv:2506.13131.

[37] NVIDIA. Nemotron 3 Nano: Open, efficient mixture-of-experts hybrid Mamba-transformer model for agentic reasoning, 2025. arXiv preprint arXiv:2512.20848.

[38] NVIDIA, A. Blakeman, A. Grattafiori, et al. NVIDIA Nemotron 3: Efficient and open intelligence, 2025. arXiv preprint arXiv:2512.20856.

[39] NVIDIA, A. Chandiramani, et al. Nemotron 3 Super: Open, efficient mixture-of-experts hybrid Mamba-transformer model for agentic reasoning, 2026. arXiv preprint arXiv:2604.12374.

[40] K. Opsahl-Ong, M. J. Ryan, J. Purtell, et al. Optimizing instructions and demonstrations for multi-stage language model programs, 2024. arXiv preprint arXiv:2406.11695.

[41] C. Packer, S. Wooders, K. Lin, et al. MemGPT: Towards LLMs as operating systems, 2023. arXiv preprint arXiv:2310.08560.

[42] J. S. Park, J. C. O'Brien, C. J. Cai, et al. Generative agents: Interactive simulacra of human behavior, 2023. arXiv preprint arXiv:2304.03442.

[43] R. Pérez-Mohedano, N. Letzelter, C. Amador, C. T. VanderRoest, and S. Bakalis. Positron emission particle tracking (PEPT) for the analysis of water motion in a domestic dishwasher. *Chemical Engineering Journal*, 259:724–736, 2015.

[44] R. Pérez-Mohedano, N. Letzelter, and S. Bakalis. Integrated model for the prediction of cleaning profiles inside an automatic dishwasher. *Journal of Food Engineering*, 196:101–112, 2017.

[45] X. Puig, K. Ra, M. Boben, J. Li, T. Wang, S. Fidler, and A. Torralba. VirtualHome: Simulating household activities via programs. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8494–8502, 2018.

[46] C. Qian, W. Liu, H. Liu, et al. ChatDev: Communicative agents for software development, 2023. arXiv preprint arXiv:2307.07924.

[47] M. Robeyns, M. Szummer, and L. Aitchison. A self-improving coding agent, 2025. arXiv preprint arXiv:2504.15228.

[48] B. Romera-Paredes, M. Barekatain, A. Novikov, et al. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.

[49] M. Savva, A. Kadian, O. Maksymets, Y. Zhao, E. Wijmans, B. Jain, J. Straub, J. Liu, V. Koltun, J. Malik, D. Parikh, and D. Batra. Habitat: A platform for embodied AI research. In *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9338–9346, 2019.

[50] N. Shinn, F. Cassano, E. Berman, et al. Reflexion: Language agents with verbal reinforcement learning, 2023. arXiv preprint arXiv:2303.11366.

[51] M. Shridhar, J. Thomason, D. Gordon, Y. Bisk, W. Han, R. Mottaghi, L. Zettlemoyer, and D. Fox. ALFRED: A benchmark for interpreting grounded instructions for everyday tasks. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10737–10746, 2020.

[52] J. Skalse, N. H. R. Howe, D. Krasheninnikov, and D. Krueger. Defining and characterizing reward hacking, 2022. arXiv preprint arXiv:2209.13085.

[53] C. Snell, J. Lee, K. Xu, and A. Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters, 2024. arXiv preprint arXiv:2408.03314.

[54] J. Snoek, H. Larochelle, and R. P. Adams. Practical Bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems 25 (NIPS 2012)*, 2012.

[55] S. Srivastava, C. Li, M. Lingelbach, R. Martín-Martín, F. Xia, K. Vainio, Z. Lian, C. Gokmen, S. Buch, C. K. Liu, S. Savarese, H. Gweon, J. Wu, and L. Fei-Fei. BEHAVIOR: Benchmark for everyday household activities in virtual, interactive, and ecological environments. In *Proceedings of the 5th Conference on Robot Learning (CoRL)*, volume 164 of *Proceedings of Machine Learning Research*, pages 477–490, 2022.

[56] I. Voysey, T. George Thuruthel, and F. Iida. Autonomous dishwasher loading from cluttered trays using pre-trained deep neural networks. *Engineering Reports*, 3(5):e12321, 2021.

[57] A. Wald. Sequential tests of statistical hypotheses. *The Annals of Mathematical Statistics*, 16(2): 117–186, 1945.

[58] F. Wang and K. Hauser. Stable bin packing of non-convex 3D objects with a robot manipulator. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 8698–8704, 2019.

[59] G. Wang, Y. Xie, Y. Jiang, et al. Voyager: An open-ended embodied agent with large language models, 2023. arXiv preprint arXiv:2305.16291.

[60] Z. Z. Wang, J. Mao, D. Fried, and G. Neubig. Agent workflow memory, 2024. arXiv preprint arXiv:2409.07429.

[61] Q. Wu, G. Bansal, J. Zhang, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation, 2023. arXiv preprint arXiv:2308.08155.

[62] C. Yang, X. Wang, Y. Lu, et al. Large language models as optimizers, 2023. arXiv preprint arXiv:2309.03409.

[63] X. Yin, X. Wang, L. Pan, et al. Gödel agent: A self-referential agent framework for recursive self-improvement, 2024. arXiv preprint arXiv:2410.04444.

[64] M. Yuksekgonul, F. Bianchi, J. Boen, et al. TextGrad: Automatic "differentiation" via text, 2024. arXiv preprint arXiv:2406.07496.

[65] E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman. STaR: Bootstrapping reasoning with reasoning, 2022. arXiv preprint arXiv:2203.14465.

[66] E. Zelikman, E. Lorch, L. Mackey, and A. T. Kalai. Self-taught optimizer (STOP): Recursively self-improving code generation, 2023. arXiv preprint arXiv:2310.02304.

[67] J. Zhang, J. Xiang, Z. Yu, et al. AFlow: Automating agentic workflow generation, 2024. arXiv preprint arXiv:2410.10762.

[68] J. Zhang, S. Hu, C. Lu, et al. Darwin gödel machine: Open-ended evolution of self-improving agents, 2025. arXiv preprint arXiv:2505.22954.

[69] A. Zhao, D. Huang, Q. Xu, et al. ExpeL: LLM agents are experiential learners, 2023. arXiv preprint arXiv:2308.10144.

[70] H. Zhao, Q. She, C. Zhu, Y. Yang, and K. Xu. Online 3D bin packing with constrained deep reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 741–749, 2021.

[71] L. Zheng, W.-L. Chiang, Y. Sheng, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena, 2023. arXiv preprint arXiv:2306.05685.

[72] M. Zhuge, W. Wang, L. Kirsch, et al. Language agents as optimizable graphs, 2024. arXiv preprint arXiv:2402.16823.